

Beyond Twelve Tips: What an Agentic Coding Assistant Catches in Applied Transport Modelling

Marco van Burgsteden*

Department / Institute, placeholder

email@placeholder

Abstract

Recent guidance on AI-assisted scientific coding – most notably Bridgeford et al.’s twelve tips for AI-assisted coding in science – treats the large language model (LLM) as a coding instrument: it writes functions, it writes tests, and the researcher remains responsible for correctness. This framing fits code-centric science well, but it understates what changes when an *agentic* coding assistant also manages the development workflow itself – running tests, driving version control, and flagging problems unprompted – in a discipline where correctness is not only a property of code but a property of *fidelity to a published specification*. We report on a short case drawn from reconstructing a legacy transport-accessibility model (IKOB) as a tested, documented Python package under a specification-first, test-driven, agentic workflow. We distinguish three classes of issue an agentic assistant surfaced during this work – correctness-class, governance-class, and specification-fidelity-class – and argue that only the first two are addressed by existing AI-assisted-coding guidance. The third, where an agent flags that the *implemented model family* silently diverges from the *specified* one, is specific to applied modelling disciplines and is, to our knowledge, not yet reported. We propose a minimal addition to existing AI-coding checklists for modelling research: an explicit, agent- or human-checked correspondence table between paper equations and the code objects that implement them.

Keywords: AI-assisted coding; agentic LLMs; transport accessibility modelling; reproducibility; research software

1 Introduction

Guidance on using large language models (LLMs) for scientific software has matured quickly. Bridgeford et al. [2026] distil twelve practical tips – organised around problem preparation, context management, testing, and code quality – for using AI coding tools without sacrificing scientific rigor. Their framing, like most of the surrounding literature [Sandve et al., 2013, Wilson et al., 2014, Pol-drack, 2024], treats the LLM as an instrument for *writing and reviewing code*: a tool whose output the researcher must independently verify against tests, domain knowledge, and critical reading.

This framing is incomplete for a workflow that is increasingly common: an *agentic* coding assistant (e.g. Claude Code, Codex) that does not just write functions on request but also runs the test suite, drives git and continuous-integration workflows, monitors its own output, and – in our experience – proactively flags things the researcher did not ask it to check. Bridgeford et al. [2026] acknowledge this shift (their Tip 9, “monitor progress and know when to restart”, and Section 3.3, “guardrails for autonomous agents”), but their treatment stays at the level of code correctness and

*Corresponding author.

workflow safety. Applied modelling disciplines such as transport geography have a third failure mode that generic coding guidance does not cover: a piece of code can be internally correct, fully covered by passing tests, and still implement the *wrong model*, because unit tests check behaviour, not correspondence to a published equation.

We report a short case from rebuilding a legacy Dutch accessibility model (IKOB) as a specified, tested, documented Python package, under a protocol combining (i) a proven specification and codebase as the starting point, (ii) specification-first, test-driven development, and (iii) an agentic coding assistant used throughout, including for run management and version control – not only for writing functions. Over the course of this work, the assistant surfaced issues in three distinct classes, summarised in Table 1. We argue that the third class is the one with no existing home in the AI-assisted coding literature, and that it matters specifically *because* the discipline’s object of concern is not the code but the model the code is supposed to represent.

2 Setting

The codebase under development reconstructs a competition-adjusted gravity accessibility model (Hansen/Shen-style) with population segmentation by income and household type, and a joint time–money tolerance filter composed via a survival copula – the implementation vehicle for a companion methodological paper [van Burgsteden et al., under review]. Development proceeded under an explicit protocol: begin from a stated mathematical specification and a previously working legacy codebase; rewrite incrementally with an agentic LLM assistant, pairing every behavioural change with a unit test before or alongside the change; keep a continuous, version-controlled record of what changed and why (commit messages, pull requests, and an explicit decision log). The assistant was given broad but directory-scoped permissions: it could read and edit the repository, run the test suite, and manage git operations, but version-control history was always inspected before any destructive operation, in line with Bridgeford et al.’s guardrail recommendations (§1; Bridgeford et al., 2026, §3.3).

3 A typology of catches

The first two classes in Table 1 are reassuring in a specific sense: they show that an agentic assistant, used under a test-driven and version-control-disciplined protocol, behaves essentially as Bridgeford et al. [2026] describe – useful, occasionally risky, and manageable with ordinary software-engineering guardrails. The third class is different in kind, not degree. A copula family is a modelling choice with first-order consequences for a paper’s conclusions (in our case, the policy interpretation of a dependence parameter θ across income segments), yet nothing in a conventional software test suite is positioned to catch its silent substitution: the code was not *broken*, it was *unfaithful*, and unfaithfulness of this kind produces no stack trace.

4 Discussion

Why did this catch happen at all? Not because the assistant was specifically instructed to audit model fidelity, but because it had simultaneous access to both the governing paper and the codebase across an extended working session, and treated a mismatch between the two as worth surfacing even unprompted – a behaviour closer to a junior collaborator cross-checking a derivation than to a code-completion tool. This is precisely the capability that distinguishes an agentic assistant used across an entire modelling workflow from the code-generation-on-request pattern that Bridgeford et

Table 1: Three classes of issue surfaced by the agentic assistant during development, and their treatment in existing AI-assisted-coding guidance.

Class	What happened	Covered by existing guidance?
Correctness	A numeric-column auto-detection routine crashed on real-world input because a pandas/GDAL nullable dtype is not a plain numpy dtype. A regression test reproducing the exact failure was written before the fix, confirmed to fail on the old code, and confirmed to pass after.	Yes – directly matches Tip 7 (Bridgeford et al.): use a failing test to specify and guard against a known bug.
Governance	A merged pull request’s branch diverged from <code>main</code> after a local IDE sync, triggering a stop-hook warning about unpushed commits. Before force-pushing to reconcile, branch and <code>main</code> contents were diffed to confirm no work would be discarded, and the GitHub API was queried to confirm the PR had in fact merged rather than vanished.	Yes – matches the guardrails in §3.3 of Bridgeford et al. (commit before changes, inspect history before anything destructive), and standard version-control practice predating AI coding entirely.
Specification fidelity	While answering an unrelated question, the assistant noted that the dependence structure implemented in the copula-composition module (a Frank copula) did not match the one specified in the governing methodological paper (a Gumbel–Hougaard survival copula). Both are valid copula families and the code’s unit tests – which check the Fréchet–Hoeffding bounds and other family-agnostic invariants – passed regardless of which family was implemented.	No – not addressed by Bridgeford et al. or related guidance, which treat correctness as a property of code behaviour rather than of correspondence to a specific published equation.

al. [2026] and most of the surrounding literature implicitly assume. It is also, we think, why the guidance in that literature under-covers the risk: a specification-fidelity failure is invisible to any checklist item framed in terms of tests, documentation, or code review, because all three operate *within* the code’s own behavioural contract, not against the paper the code is meant to implement.

We do not think this generalises to a claim that agentic assistants reliably catch specification drift – our evidence is a single case, surfaced opportunistically rather than systematically, and absence of further catches in this project is not evidence of their absence elsewhere in the code. The actionable claim is narrower: applied modelling disciplines should not assume that the existing AI-assisted coding checklists (document the workflow, test the code, review the diff) are sufficient, because none of their items are built to catch this failure mode, and a modelling paper’s validity rests on it more than most scientific software does.

4.1 A minimal addition to existing guidance

We propose a small, low-cost extension to Bridgeford et al.’s Tip 12 (documentation for reuse and publication), specific to quantitative modelling papers: alongside the existing checklist items (work-flow documentation, docstrings, environment specification, README, worked examples), maintain an explicit *equation–code correspondence table* mapping each numbered equation in the methodological paper to the function, class, or parameter that implements it, and have either a human co-author or the coding assistant itself check this table against the current state of the code as a discrete, named step – not folded silently into general code review. This does not require new tooling; it requires only that fidelity-checking be made a visible, repeatable step rather than an incidental by-product of a long working session, as it was in our case.

5 Conclusion

Existing guidance for AI-assisted scientific coding treats correctness as a property of code. In applied transport and geographic modelling, correctness is also a property of a model’s fidelity to its own specification, and this is a failure mode that current AI-coding checklists do not address. An agentic assistant working across an entire modelling workflow – not just generating functions on request – is positioned to catch this failure mode in a way that purely code-centric tooling is not, but only if researchers treat specification fidelity, not just test coverage, as something to actively check for. We offer the typology in Table 1 and the correspondence-table proposal in §4 as a starting point for a more systematic treatment.

AI usage disclosure

An agentic LLM coding assistant (Claude Code, Anthropic) was used throughout the development of the codebase discussed in this note, including for code authorship, test authorship, version-control operations, and – as discussed in §3 – for surfacing the specification-fidelity issue reported here. The assistant was also used in drafting this manuscript. All code, tests, and commit history are available at the project repository (placeholder: insert public URL at submission time, if the repository is made public; it is currently private).

Data and code availability

Code: https://github.com/Marrekoo/IK0B_Aporometrica (currently private; visibility to be confirmed before submission).

References

- Bridgeford EW, Campbell ID, Chen Z, Lin Z, Ritz H, Vandekerckhove J, Poldrack RA. Twelve quick tips for AI-assisted coding in science. *PLOS Computational Biology* 2026;22(7):e1014428. <https://doi.org/10.1371/journal.pcbi.1014428>
- Sandve GK, Nekrutenko A, Taylor J, Hovig E. Ten simple rules for reproducible computational research. *PLOS Computational Biology* 2013;9(10):e1003285.
- Wilson G, Aruliah DA, Brown CT, et al. Best practices for scientific computing. *PLOS Biology* 2014;12(1):e1001745.

Poldrack RA. Better code, better science. 2024. <https://poldrack.github.io/BetterCodeBetterScience/>

van Burgsteden M, Geurs KT, Grigolon AB, Ulak MB. Either you can reach it or you cannot: threshold microfoundations for accessibility, applied to shared bicycle provision in Utrecht. Under review.